# Conceptual Architecture of WezTerm
## Coffee Lovers

October 9th, 2026

Group #20
Daniel Asayesh: 20477703 23hmh5@queensu.ca
Mahuel Bunsie: 20457606 23sfhj@queensu.ca
Jashan Uppal: 20454134 jashan.u@queensu.ca
Aariz Anish Bhamani: 20452073 bhamani.aarizanish@queensu.ca
Lovepreet Singh: 20498604 singh.lovepreet@queensu.ca
Navtej Singh Bawa: 20451254 23vth2@queensu.ca

---

## Abstract

This report examines the conceptual architecture of WezTerm, an open-source, cross-platform terminal emulator and multiplexer written in Rust. WezTerm provides a graphical environment for running shells and command-line applications, with support for multiple windows, tabs, split panes, searchable scrollback and local or remote sessions.

WezTerm consists of several major components with separate responsibilities. The graphical interface handles user interaction and displays terminal content, while the multiplexer manages windows, tabs, panes and terminal sessions.Terminal processing interprets application output, and pseudo-terminals (PTYs) provide communication between WezTerm and running programs. These components work together to process user input and display application output while keeping session management separate from graphical presentation.

The architecture can be understood as largely layered and event-driven, with concurrency allowing multiple terminal sessions and activities to make progress during the same period. The separation between terminal processing, session management, graphical presentation, and operating-system functionality has also supported WezTerm's evolution. Features such as different rendering options, remote multiplexing, and Lua-based customization have been introduced while maintaining these general architectural boundaries.

These boundaries provide a natural division of responsibilities for developers working on terminal behaviour, rendering, multiplexing, networking, and configuration. The sequence diagrams examine how the components interact when running a command and splitting a pane to start another shell. A major strength identified in WezTerm's architecture is its ability to support different platforms and session types while allowing individual components to evolve without requiring changes to the entire system.

**Table of Contents:**

**Abstract**

**1. Introduction & Overview**

**2. System Overview**

- **2.1 What WezTerm Is**
- **2.2 Major Components**
- **2.3 Component Interactions**

**3. System Evolution**

**4. Control Flow & Data Flow**

- **4.1 Control Flow**
- **4.2 Data Flow**

**5. Concurrency**

**6. Developer Responsibilities & Component Ownership**

- **6.1 Terminal Behaviour and Graphical Display**
- **6.2 Sessions, Processes, and Connections**
- **6.3 Customization and User Support**
- **6.4 Shared Responsibilities**

**7. Sequence Diagrams**

- **7.1 Running a Command**
- **7.2 Splitting a Pane and Starting a New Shell**

**8. Conclusion**

**References**

**Appendix A: AI Collaboration Report**

## Introduction and Overview

WezTerm is an open-source, cross-platform terminal emulator and multiplexer written in Rust. It provides users with an environment for running shells and command-line applications while supporting features such as multiple windows, tabs, split panes, and remote terminal sessions. Although these features are accessed through a graphical interface, they rely on several components working together to manage user input, terminal sessions, and communication with running programs.

The purpose of this report is to examine WezTerm's conceptual architecture and understand how its major components interact. The focus is on identifying the responsibilities of these components, how information moves between them, and how the architecture supports the system's functionality. Rather than examining individual functions or implementation details, the report uses available documentation and project resources to develop a higher-level understanding of WezTerm's design.

WezTerm's architecture can be understood through the separation of its graphical interface, multiplexer, terminal processing, and pseudo-terminal (PTY) functionality. The graphical interface handles user interaction and displays terminal content, while the multiplexer organizes terminal sessions into windows, tabs, and panes. Terminal processing interprets application output, and the PTY provides communication between WezTerm and running programs. This separation allows terminal sessions to be managed independently from the graphical interface, which is particularly important for local and remote multiplexing.

The report begins with an overview of WezTerm's major components and their interactions, followed by an examination of how the system has evolved. It then discusses control flow and data flow to explain how user actions are processed and how information travels through the system. Concurrency is also examined to understand how WezTerm supports multiple running sessions and processes terminal output while other activities continue. The report considers how the architecture influences the division of responsibilities among developers and uses two sequence diagrams to demonstrate the interactions involved in running a command and splitting a pane to start another shell.

The main finding of this analysis is that separating terminal session management from graphical presentation is an important strength of WezTerm's architecture. Along with event-driven behaviour and concurrent processing, this separation allows the system to support multiple sessions and different operating environments. It has also provided flexibility as WezTerm has evolved through the addition of new rendering options, remote multiplexing, and Lua-based customization. However, the components still depend on one another, making coordination and consistent communication between them important when maintaining or extending the system.

## System Overview

### What WezTerm Is

WezTerm is a cross-platform terminal emulator and multiplexer written in Rust. It gives users a graphical window in which to run shells and command-line programs, and it runs on Linux, macOS, Windows 10, FreeBSD and NetBSD. It does more than a basic terminal window. It can manage multiple windows, tabs and split panes, it supports scrollback with search, and it includes an SSH client with native tabs. It can also multiplex panes, tabs and windows on both local and remote hosts.

### Major Architectural Components

**Graphical user interface:** The graphical interface is the part of WezTerm that the user interacts with directly. Like any terminal emulator, it reads the output of the program it runs and interprets the escape sequences in it to produce a display. It also decodes keyboard and mouse input from the windowing environment and encodes it for the running program. WezTerm's documentation treats the interface and the multiplexer as separate concerns. The multiplexer may not be connected to a GUI at all, so operations that need a window system are not available in the interface it exposes to scripts. Managing terminal sessions and displaying them are therefore different responsibilities.

**Multiplexer:** The multiplexer, often called the mux, organizes the running programs into panes, tabs, windows and workspaces. WezTerm can also create multiplexing domains, and each domain is a distinct set of windows and tabs. Domains can be local or reached through a Unix socket, SSH or TLS, which is how the same model covers both local and remote sessions.

**Pane:** A pane holds a single terminal session inside a tab. Through WezTerm's Pane object, the multiplexer exposes the pane's screen contents and cursor position, its dimensions, the name of its TTY, the foreground process running in it, and a way to send text to that process. A pane is therefore the point where the running program and what WezTerm displays meet.

**Pseudo terminal (PTY):** The PTY is the connection between WezTerm and the program it runs. It is the software evolution of the older physical terminal connection, which let a user-space application define virtual terminals and pass along information such as window size changes. A terminal emulator creates a PTY and then starts a child program, usually a shell, inside it. On Windows, a helper program is also started to translate between escape sequences and the native console, using ConPTY.

**Shell:** A shell, such as bash or zsh, is a program that lets the user navigate the computer system and launch other programs. It communicates with the terminal indirectly, through the PTY.

**Applications:** Applications are the command-line programs the shell starts, for example a text editor such as vim. The shell passes them the terminal's input and output streams, and from then on they send data to the terminal directly, without the shell taking part in the transfer.

### Component Interactions

At a high level, the parts form a chain:

User - WezTerm GUI - Multiplexer/Pane - PTY - Shell/Application

When the user types, WezTerm receives the keystroke from the windowing environment, encodes it, and directs it to the active pane. The pane writes it into the PTY, and the shell or application at the other end reads it and acts on it. Any output travels back through the PTY to WezTerm, which interprets the escape sequences in it and draws the result in the graphical interface. The same chain applies to a remote session, with the multiplexer reaching the PTY through a domain instead of a local process.

## System Evolution

WezTerm is a GPU-accelerated, cross-platform terminal emulator and multiplexer written in Rust by Wez Furlong, who remains its author [2]. The project began in early 2018, when its GitHub repository was created on February 7 [1].

**2018 - 2019:** The earliest tagged releases we found date from March 2019, and they consist mostly of changes to the continuous-integration services that build and package releases, including Debian packages [2]. The project also keeps a "Nightly Builds" release that holds builds of whatever is on the main branch at the time, with a warning that these may be unstable [2]. By June 2019, the release notes were already linking to documentation on multiplexing, and a nightly pre-release was scheduled [2]. The release of November 2019 was a large step. WezTerm gained a tab bar, hot-reloading of the configuration file, the wezterm ssh command with SSH multiplexer sessions, serial-port connections, inline image display, input-method support on macOS and Windows, and an automatic fallback to software rendering when no GPU is available [2]. A month later it added native Wayland support. It also split its single font setting into separate locator, shaper and rasterizer options, which made the font pipeline easier to change piece by piece [2].

**2020:** In May 2020, WezTerm began checking GitHub once a day for newer stable releases and showing a notice to users, and it became installable on Windows through Scoop [2]. The October 2020 release was a turning point. The Lua scripting interface was expanded so that scripts could react to URI clicks and keyboard events through wezterm.oncallbacks. The multiplexer server was moved out of the main program into its own wezterm-mux-server executable, which forced users to revise their serve_command setting. A fix for bootstrapping certificates in TLS multiplexing was also included [3]. Some changes in this release broke existing configurations. The GUI began normalizing the Shift key state, so a key binding written with CTRL|SHIFT had to be rewritten after updating [3].

**2021 - 2023:** Tagged releases continued through these years. In June 2022, the wezterm.mux module brought the multiplexer layer into Lua, and its documentation notes that the multiplexer may run without a GUI attached [4]. In November 2022, WezTerm added WebGpu as a third rendering option, next to OpenGL and Software, with Metal, Vulkan and DirectX 12 as possible back ends [5]. In 2023, there were a handful of releases between March and July [2].

**2024 - Present:** On January 27, 2024, the default renderer was switched to WebGpu, and the following day's release set it back to OpenGL [5]. WezTerm also falls back to Software rendering on its own in a Windows Remote Desktop session [5]. The release of February 3, 2024 is listed as the latest on the project's releases page [2]. Since then, development has continued mainly through nightly builds, and users have asked for a new stable release [6].

Up to now, WezTerm is still under active development. Looking across this history, new features were usually added next to the old ones, such as a third renderer or a separate server program. Breaking changes were announced in the release notes, and mistakes were corrected within days. We believe that the way the project is divided into separate parts is one reason it can keep changing, because a change in one part, such as the renderer or the multiplexer server, does not require rewriting the others [3], [5].

## Control Flow & Data Flow

### Control Flow

WezTerm mainly follows an event-driven control flow. In other words, a lot of what happens depends on an event from the user or from the terminal session itself. For example, when the user presses a key, WezTerm first has to determine what that key press means. It could be a shortcut that WezTerm handles itself, such as opening a new tab or splitting a pane, or it could simply be input that needs to be sent to the program running in the terminal. The WezTerm documentation describes this process through its key assignment system: configured key combinations can trigger WezTerm actions, while other input can be passed on to the terminal [5], [7].

An important part of this control flow is the multiplexer, usually referred to as the mux. The mux is responsible for managing terminal-related objects such as windows, tabs, panes, and workspaces. At first it is easy to think of these as being controlled directly by the GUI, but WezTerm actually separates the GUI and mux layers. The documentation notes that the mux can exist without a GUI being attached to it [4]. This is important because it means the terminal sessions are not completely dependent on what is currently being displayed in a graphical window.

For a normal local terminal session, the control flow can be simplified as:

**User - GUI/Input Handling - Multiplexer - Pane - PTY - Shell/Application**

The PTY, or pseudo-terminal, is what connects the terminal emulator to the shell or other command-line program. When a shell is started, it runs on the other side of the PTY. WezTerm receives the user’s input and sends the appropriate data through the terminal connection to that program. The program can then respond with its own output [1]. So, although the user sees a single terminal window, there are several pieces involved in controlling what happens after an input event.

**Data Flow**

The data flow is closely related to the control flow, but it focuses more on what information is actually moving through the system. In WezTerm, the main flow is two-way. Input moves from the user toward the shell or application, and the output from that application moves back through WezTerm so it can eventually be displayed to the user [1].

A simplified input path is:

**User - Keyboard/Mouse Input - WezTerm - Pane - PTY - Shell/Application**

For example, if a user types a command such as `ls`, the keyboard input is received by WezTerm. Assuming those keys are not being used for one of WezTerm's own shortcuts, they are encoded as terminal input and sent through the active terminal session to the program running inside it [1], [7].

The output path works in the opposite direction:

**Shell/Application - PTY - Pane/Terminal Emulator - Screen State - GUI/Renderer - User**

The data coming back from an application is not always just plain text that can immediately be drawn on the screen. Terminal programs can also send control or escape sequences. These sequences can tell the terminal to do things such as change colours, move the cursor, change text formatting, or update other parts of the terminal state [1], [6]. WezTerm therefore has to interpret the output before the final result is presented to the user.

The pane is also more than just the rectangular area that the user sees on the screen. WezTerm's documentation shows that a pane is associated with the terminal session and exposes information about the terminal state and scrollback, while also allowing input to be sent to the process running inside it [3]. This makes the pane an important point between the running program and the information that WezTerm eventually presents.

There can also be additional data passed from the shell to WezTerm. Through shell integration, applications can use supported escape sequences to communicate information such as the current working directory and the locations of prompts, commands, and command output. WezTerm can then use this information for features such as spawning a new pane in the current directory or navigating through command output in the scrollback [8].

The difference between the two is that control flow explains which parts of WezTerm are responsible for deciding and carrying out an action, while data flow explains the information moving between those parts. The separation between the GUI and mux is especially useful here. Since the mux manages terminal sessions separately from the GUI, WezTerm can use a similar model for both local and remote terminal sessions. Its multiplexing documentation describes domains that can contain windows, tabs, and panes, with the GUI attaching to those domains when needed [2], [4].

**Concurrency**

Concurrency in WezTerm means that several activities can make progress during the same period. A user may be entering commands while another program produces output or a remote session continues running. These activities do not all have to execute at exactly the same instant. What matters is that one activity can continue without requiring every other activity to finish first.

One visible example is the use of multiple terminal sessions. WezTerm organizes sessions into panes, tabs and windows through its multiplexer, usually called the mux. A user could run a Python program in one pane while checking Git changes in another. The operating system schedules the programs, while WezTerm manages their terminal connections and displays their output. This allows users to work with several running programs through the same application.

Concurrency also occurs inside WezTerm when it handles program output. Local programs communicate with the terminal through pseudo-terminals or PTYs. Reading from a PTY can involve waiting until the program sends more data. WezTerm performs this reading in background threads and passes the received output to parser threads. The parsers interpret both text and terminal instructions, such as requests to move the cursor or change colours. The parsed actions update the appropriate pane's terminal state, and WezTerm sends a notification that the pane has new output [9].

This separation helps explain how WezTerm can remain responsive while programs are running. Reading output happens in a separate thread from parsing it and updating the pane. This separation can help the interface remain responsive while terminal programs are running [9]. For example, a program might pause before printing its next result, while the user continues working in another session. This is an architectural benefit of separating these tasks, although it does not guarantee that the interface can never slow down under heavy workloads.

The different threads still need to coordinate their work. If two sessions produce output during the same period, each output stream must remain associated with its own pane. Instructions within a stream must also retain their order so that cursor movements and text updates produce the intended result. WezTerm passes information between threads and uses synchronization when accessing shared mux data. Concurrent processing therefore works alongside controlled access to session information.

There is also concurrency across separate processes. WezTerm can connect its graphical interface to a multiplexer running locally through a Unix-domain socket or remotely through SSH or TLS. In these arrangements, the graphical client and multiplexer run as separate processes, potentially on different computers. A user could interact with a local shell while a remote program continues its work. Unix-domain communication is usually local, so it should be distinguished from communication across a network.

Lua customization introduces another consideration. WezTerm's maintainer explains that different threads and execution contexts may use separate Lua states. As a result, developers should not assume that an ordinary Lua global variable is shared by every callback. For example, storing information about one window in a global variable may not make that information

available wherever another callback runs. The maintainer recommends using the window passed to the callback when working with that window.

The configuration file may also be evaluated repeatedly during startup or reloading. This is not, by itself, evidence that those evaluations happen concurrently. However, it matters when writing scripts with side effects. Launching a background process directly in the configuration could start additional copies each time the file runs. Developers therefore need to consider both where their Lua code executes and whether its actions can be repeated safely [11].

## Developer Responsibilities & Component Ownership

WezTerm has several components with different responsibilities. Its source repository separates areas such as terminal emulation, the graphical interface, the multiplexer, process communication, and configuration [10]. These boundaries provide a useful way to divide development work. Developers could focus on particular components while learning how those components connect to the rest of the system. The division described here is based on the architecture, rather than WezTerm's actual contributor assignments.

### Terminal behaviour and graphical display

Developers working on the terminal core could take responsibility for interpreting program output and maintaining terminal state. This includes handling escape sequences, cursor movements, text formatting, and scrollback [1]. They would need to understand terminal behaviour and test compatibility with command-line applications. Their work also requires coordination with GUI developers because terminal state determines what appears on the screen. For example, an incorrect cursor position could cause text to appear in the wrong place even when the renderer draws it correctly.

GUI and rendering developers could focus on drawing text, displaying fonts and colours, and handling keyboard and mouse input. Platform specialists could maintain window behaviour and operating-system integration. WezTerm's repository includes separate areas for the GUI, fonts, and window support [10]. These developers would need to agree with terminal-core developers on how screen information is represented and displayed.

### Sessions, processes, and connections

The multiplexer, or mux, provides another area of responsibility. Developers working on it could manage panes, tabs, windows, workspaces, and the organization of local or remote sessions [2]. Creating a split pane shows how this work connects to other components:

- The process layer starts a shell and establishes its terminal connection.
- The mux organizes the new session within the tab.
- The GUI displays the updated pane layout.

The developers involved should therefore agree on what happens when a pane is created, resized, or closed.

Developers responsible for processes and pseudo-terminals could handle starting shells and maintaining communication between local programs and WezTerm. A pseudo-terminal, or PTY, connects the terminal emulator to a shell or another command-line application [1]. This work requires knowledge of operating-system differences. It also overlaps with session management: when a program exits, the process layer and mux need to handle that change consistently so that the correct session is updated or closed.

Networking developers could maintain communication with separate multiplexer processes. WezTerm supports local connections through Unix-domain sockets and remote connections through SSH or TLS [2]. Developers in this area would need experience with connection handling, protocols, and security. They should coordinate with mux and GUI developers when deciding how connection failures or reconnections affect sessions and what the user sees.

**Customization and user support**

Configuration and Lua API developers could maintain settings and the scripting interfaces used to customize WezTerm [11]. CLI developers could maintain commands that control a running GUI or multiplexer, including commands for creating panes and sending input [13]. Changes to session behaviour may affect both interfaces, so their developers need to work with the relevant component owners.

Documentation contributors could update instructions and examples, while plugin contributors could maintain reusable Lua customization code [14]. Developers changing a feature should help ensure that its documentation and examples still match its behaviour.

**Shared responsibilities**

Some responsibilities cross all these areas. Developers should agree on the information exchanged between components and discuss changes that affect those agreements. Each developer can check their own component, but testing should also cover how the components work together. Useful checks include:

- Opening, resizing, and closing a pane.
- Processing program output and displaying it correctly.
- Connecting to a remote session and handling a lost connection.
- Checking affected behaviour across operating systems.

WezTerm’s structure supports parallel development while still requiring collaboration. Clear ownership helps identify who should investigate a problem or review a change. However, developers need enough understanding of neighbouring components to recognize when their work affects others. Testing the connections between components is therefore as important as testing each component separately.

## Sequence Diagrams

The following sequence diagrams show how WezTerm's main parts work together during two common actions: running a command and splitting a pane to start another shell. They focus on the main steps and the parts involved, without going into individual functions.

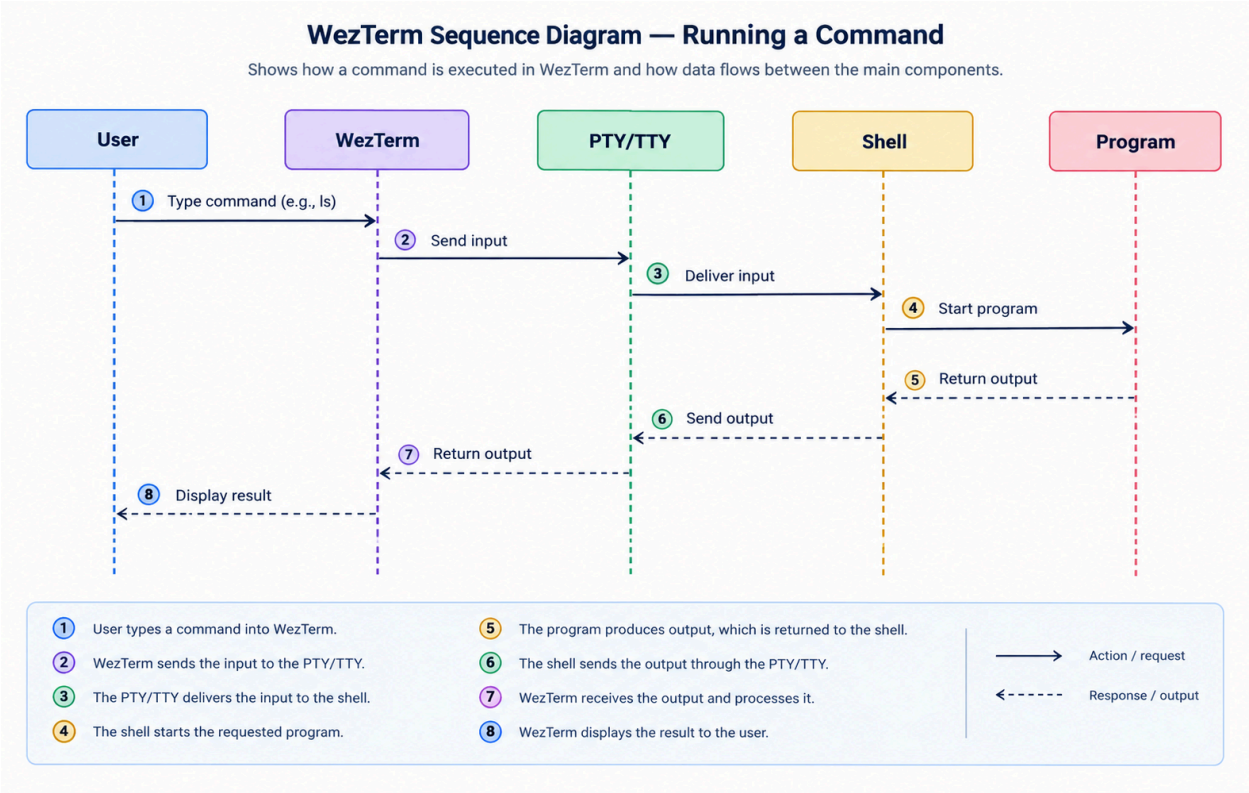



**Figure 1: Running a command**

The first diagram starts when the user types a command, such as `ls`, and presses Enter. WezTerm receives the keyboard input and checks whether it matches a configured shortcut. If it does not, WezTerm sends the input to the active terminal session [7].

The input passes through a pseudo-terminal, or PTY to the shell. The PTY connects WezTerm to the programs running inside the terminal. The shell reads the command and starts the requested program. WezTerm handles input and displays output, while the shell interprets commands and launches programs [1].

The program sends its output through the PTY to WezTerm. This output can contain text and escape sequences that tell the terminal to change colours, move the cursor or update the display. WezTerm reads these instructions, updates the terminal state, and displays the result [1].

The diagram shows how input and output move in opposite directions. Input travels from the user to the shell or running program, while output returns through the PTY to WezTerm.

Although the shell starts the program, the program normally sends its output directly through the PTY.

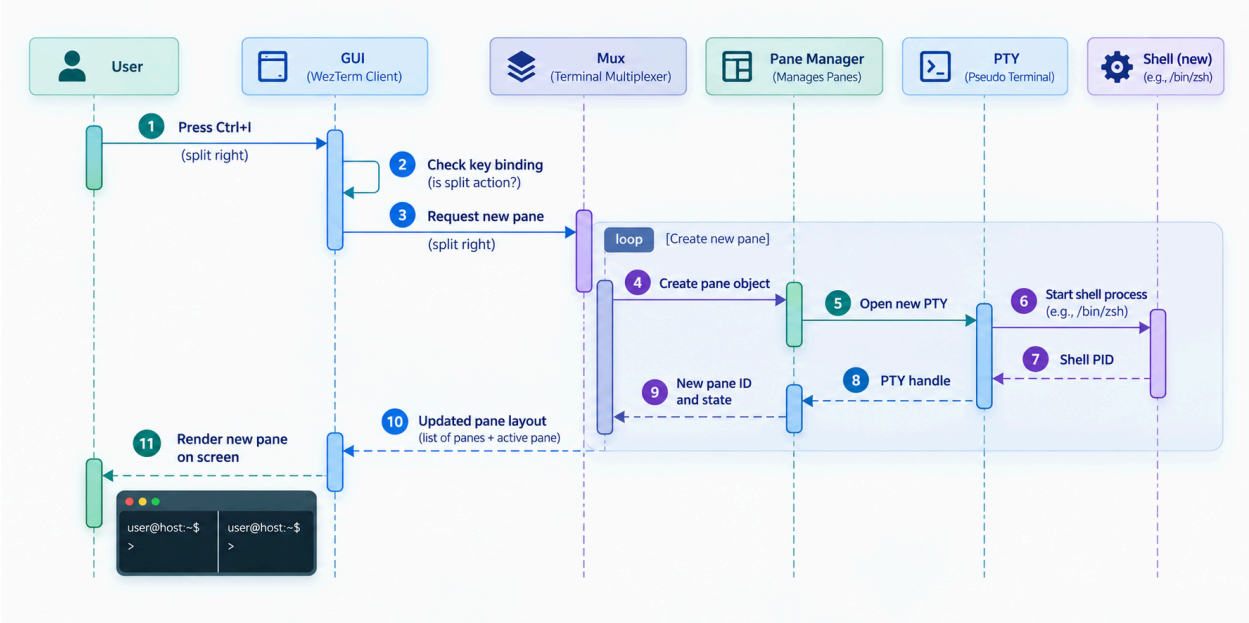



**Figure 2: Splitting a pane and starting a new shell**

The second diagram starts when the user presses a shortcut configured to split the active pane. The GUI checks the key binding and carries out the assigned WezTerm action. In this case, the shortcut changes the pane layout instead of sending input to the existing shell [7].

WezTerm then creates a new pane and starts a program inside it. The SplitPane action specifies where the pane should appear and which program it should run. If no program is specified, WezTerm uses the configured default program [5].

For the local session shown in the diagram, WezTerm creates a new PTY and starts a shell connected to it [1]. The mux organizes the new pane within the tab, and the GUI displays the changed layout. The user can then work in either pane.

The main responsibilities shown are:

- **GUI:** check the shortcut and display the new layout.
- **Mux and pane management:** organize the new pane within the tab.
- **Process and PTY support:** start the shell and connect it to WezTerm.

Splitting a pane involves more than changing what appears on the screen. The new pane also needs its own terminal connection and a running program. These parts must work together before the user can enter commands in it.

The two diagrams show different uses of the architecture. Running a command uses an existing session, while splitting a pane creates a new session and updates the layout.

**Collaboration Report:**

**AI Member Profile and Selection Process**

For this deliverable, our group used **OpenAI ChatGPT (GPT-5.6 Sol, October 2026)** as our virtual AI teammate. We chose ChatGPT because it was helpful for organizing technical information, explaining difficult software architecture ideas in simpler words, and improving the overall flow of our report.

We felt that ChatGPT was a good fit for this project because our WezTerm report included several connected topics, such as the graphical interface, multiplexer, panes, PTYs, shells, control flow, data flow, concurrency, developer responsibilities and sequence diagrams. ChatGPT helped us work through some of these relationships and explain them more clearly. We also considered that other AI tools could provide similar help but some members of our group already had experience using ChatGPT, which made it easier to use effectively. We did not treat ChatGPT as our main source of technical information. Instead, we used it as a support tool while our group remained responsible for the research, sources, and final decisions.

**Tasks Assigned to the AI Teammate**

We used ChatGPT for several parts of the project:

- **Organizing report sections:** ChatGPT helped us think about how to arrange information in sections such as the system overview, control flow, data flow, concurrency, developer responsibilities, and sequence diagrams. We felt this was a good task for AI because we already had the research and mainly needed help organizing it.
- **Explaining technical concepts:** We asked ChatGPT to help simplify ideas such as the mux, panes, PTYs, shells, and how the different WezTerm components interact. This made it easier for us to explain these concepts in a way that was clear to the reader.
- **Improving wording:** Some of our original notes were short or very technical, so we used AI to suggest clearer and more natural ways to explain them.
- **Brainstorming sequence diagrams:** ChatGPT helped us think through the main steps that happen when a user runs a command or splits a pane and starts another shell.
- **Checking clarity:** We also asked ChatGPT whether some explanations were too complicated or whether they clearly answered the project question.

We believed these were suitable tasks for AI because they involved brainstorming, explaining, organizing and editing. The group still made the final technical decisions and decided what information should actually appear in the report.

**Interaction Protocol and Prompting Strategy**

Three of the six group members worked together on the prompting process. Instead of having only one person communicate with ChatGPT, these three members discussed what they wanted

to ask before entering a prompt. They looked at the assignment requirements, our research and the section being worked on and then decided what type of answer would be most useful.

After receiving a response, the three members reviewed it and shared useful parts with the rest of the group. The group then discussed whether the response was clear, relevant, and consistent with our research. If the answer was too general, too technical, or did not match what we needed, the prompt was changed and tried again.

One example of a prompt we used was:

> "Using our WezTerm notes, explain how the GUI, multiplexer, pane, PTY, shell, and application interact when a user runs a command. Keep the explanation simple and suitable for this software architecture report. Do not add technical details that are not supported by the information provided."

We often improved our prompts by adding instructions such as "use simpler language," "focus on the main components," "make this sound more natural," or "write this as a short report section." We found that ChatGPT gave much better responses when we provided enough context and clearly explained what kind of answer we wanted.

**Validation and Quality Control Procedures**

We did not assume that everything ChatGPT gave us was correct. Any important technical information suggested by AI was checked against the WezTerm documentation and the sources we had already researched.

The three members working directly with ChatGPT first checked whether the response matched the source material. We paid particular attention to technical terms such as mux, pane, PTY, shell and terminal session. After that, the rest of the group reviewed the AI-assisted content for accuracy, clarity, tone and consistency with the rest of the report.

If we found information that we could not confirm from our sources, we either removed it, changed the wording, or researched it further. We also checked our sequence diagrams against our written explanations. For example, we made sure that the flow of user input, the role of the PTY and the responsibilities of the GUI and mux matched what we found in our research.

We also checked references ourselves instead of automatically trusting any references or claims suggested by AI. The final decision about what stayed in the report was made by the group.

**Quantitative Contribution to the Final Deliverable**

We estimate that AI contributed about 35% to the final deliverable. This is an approximate estimate since it is difficult to measure the exact amount of AI involvement.

We divided the AI contribution roughly as follows:

- Organizing report sections – 8%
- Explaining technical concepts – 10%

- Improving wording and readability – 8%
- Brainstorming sequence diagrams – 5%
- Reviewing clarity and completeness – 4%

Total estimated AI contribution: 35%

Most of the AI help was used for organizing ideas, simplifying difficult concepts, improving wording and helping us think through the sequence diagrams. The remaining 65% was completed by the group, including researching WezTerm, reading the documentation, selecting sources, checking technical accuracy, reviewing diagrams, editing the report and deciding what information should be included.

Overall, ChatGPT made some parts of the process faster and easier but the group was still responsible for understanding the system, checking the information and making the final decisions.

**Reflection on Human-AI Team Dynamics**

Using ChatGPT was useful because it gave us another way to brainstorm and organize our ideas. Having three group members work together on prompting also helped because they could discuss what they wanted from the AI before asking it. This made the prompts more specific and gave us more than one person checking the responses.

AI saved time when we already understood a concept but were struggling to explain it clearly. However, it also created some extra work because the responses still needed to be checked carefully. Sometimes an answer sounded correct but was too general or included details that we could not immediately verify.

There were also cases where the group had to discuss whether an AI suggestion should be included. In these situations, we did not simply follow the AI response. We went back to our research and made the final decision as a group.

The main thing we learned is that AI works best as a supporting teammate rather than the final decision maker. Clear prompts, group discussion and source checking were important throughout the process. In future projects, we would continue using AI for brainstorming, organization and explaining difficult ideas while keeping the final research, validation and decisions in the hands of the group.

## Conclusion

The analysis of WezTerm's conceptual architecture shows how several components work together to provide a terminal emulator that supports multiple sessions, remote connections, and customization. The graphical interface, multiplexer, terminal processing, and pseudo-terminal (PTY) functionality each have separate responsibilities, but they depend on one another to process user input and display application output correctly. One of the main strengths of this architecture is the separation between terminal session management and graphical presentation. This allows WezTerm to manage sessions independently from the graphical interface, which is particularly useful when working with local and remote terminal connections.

WezTerm's architecture also combines layered responsibilities with event-driven behaviour and concurrency. User actions are processed through the appropriate components, while information moves between the running application and graphical interface through the terminal session. Concurrency allows multiple programs and terminal sessions to make progress during the same period without requiring one session to finish before another can continue. However, this also introduces the need for coordination between components, particularly when handling terminal output, updating session information, and maintaining consistent terminal state.

The evolution of WezTerm demonstrates how its architectural structure has supported the introduction of new functionality. Features such as additional rendering options, remote multiplexing, and Lua-based customization have been added while maintaining the general separation between components. These boundaries also provide a useful way to divide responsibilities among developers, allowing different areas of the system to be maintained and extended separately. At the same time, changes to one component may affect others, making communication between developers and integration testing important.

The sequence diagrams further demonstrate how these components interact during common operations, such as running a command or splitting a pane to start another shell. Although these actions may appear simple from the user's perspective, they involve several components coordinating to complete the requested operation. This highlights the importance of clearly defined responsibilities and communication between different parts of the system.

WezTerm's conceptual architecture provides a foundation for understanding how the system operates and how its design supports continued development. Its separation of responsibilities offers flexibility, but maintaining the relationships between components remains important as new features are introduced. This analysis also provides a starting point for examining WezTerm's concrete architecture in greater detail, where the relationships identified through documentation can be compared with the actual implementation.

**Sources**

[1] WezTerm, “What is a Terminal?” WezTerm Documentation. wezterm.org/what-is-a-terminal.html
[2] WezTerm, “Multiplexing,” WezTerm Documentation. wezterm.org/multiplexing.html
[3] WezTerm, “Pane Object,” WezTerm Documentation. wezterm.org/config/lua/pane/
[4] WezTerm, “wezterm.mux Module,” WezTerm Documentation. wezterm.org/config/lua/wezterm.mux/index.html
[5] WezTerm, “SplitPane,” WezTerm Documentation. wezterm.org/config/lua/keyassignment/SplitPane.html
[6] WezTerm, “Escape Sequences,” WezTerm Documentation. wezterm.org/escape-sequences.html
[7] WezTerm, “Keyboard Concepts,” WezTerm Documentation. wezterm.org/config/keyboard-concepts.html
[8] WezTerm, “Shell Integration,” WezTerm Documentation. wezterm.org/shell-integration.html
[9] WezTerm contributors. (n.d.). *Multiplexer source code (mux/src/lib.rs)*. GitHub. https://github.com/wezterm/wezterm/blob/main/mux/src/lib.rs
[10] WezTerm contributors. (n.d.). *WezTerm source repository*. GitHub. https://github.com/wezterm/wezterm
[11] WezTerm. (n.d.). *Configuration*. WezTerm Documentation. https://wezterm.org/config/files.html
[12] Furlong, W. (2023, March 2). *Response to “Update variable with tabs count”* [Discussion comment]. GitHub. https://github.com/wezterm/wezterm/discussions/3181
[13] WezTerm. (n.d.). *wezterm cli*. WezTerm Documentation. https://wezterm.org/cli/cli/index.html
[14] WezTerm. (n.d.). *Plugins*. WezTerm Documentation. https://wezterm.org/config/plugins.html